

Mandate

Agent-to-agent B2B purchasing on Razorpay, where every money action is bounded, gated, and provable.

Built by **Dev Chalana** · Repository: github.com/dev591/razorpay.

Runs with no credentials. Both key sets are optional — without an OpenAI key the agents quote from the rule-based path and every cart is labelled degraded; the gates, floors, shortlist, hash lock and audit chain all work unchanged. `cp .env.example .env` and start it.

~6,100

LINES PYTHON · 5
LAYERS

~6,600

LINES TS/REACT · 5
ROUTES

30

HTTP ENDPOINTS

5

PURPOSE-BUILT DATA
STRUCTURES

765

REAL NEGOTIATIONS
IN CORPUS

01 The Thesis

An agent that can spend money is only useful if it can also be **stopped, bounded, and audited**. Most agentic-commerce demos show an LLM producing a plausible order. The hard part isn't producing the order — it's being able to say, afterwards, exactly what was agreed, why, under what limits, and prove none of it changed.

The model proposes, the protocol disposes.

Every constraint that protects money is arithmetic in code, not an instruction in a prompt. An adversarial, hallucinating, or entirely absent model cannot breach any of them. Three consequences shape the whole system:

1. **Determinism beats persuasion.** Margin floors, budget ceilings, delivery windows and basket contents are checked in code before a model output is allowed to influence anything.
2. **The record is the product.** A hash-chained, append-only ledger means the audit trail cannot be quietly edited — including by the person who built it.
3. **Degraded is a first-class state, not a failure.** The system completes real orders with the LLM entirely unreachable, and labels every such cart.

02 Architecture

Five layers, one dependency rule: `protocol/` never imports from `agents/` or `orchestrator/`. It's the layer that must stay trustworthy independent of what any agent does — where a protocol-level check genuinely needs catalog data (budget feasibility, basket availability), the import happens lazily *inside the function*, specifically to avoid inverting that direction at module scope.

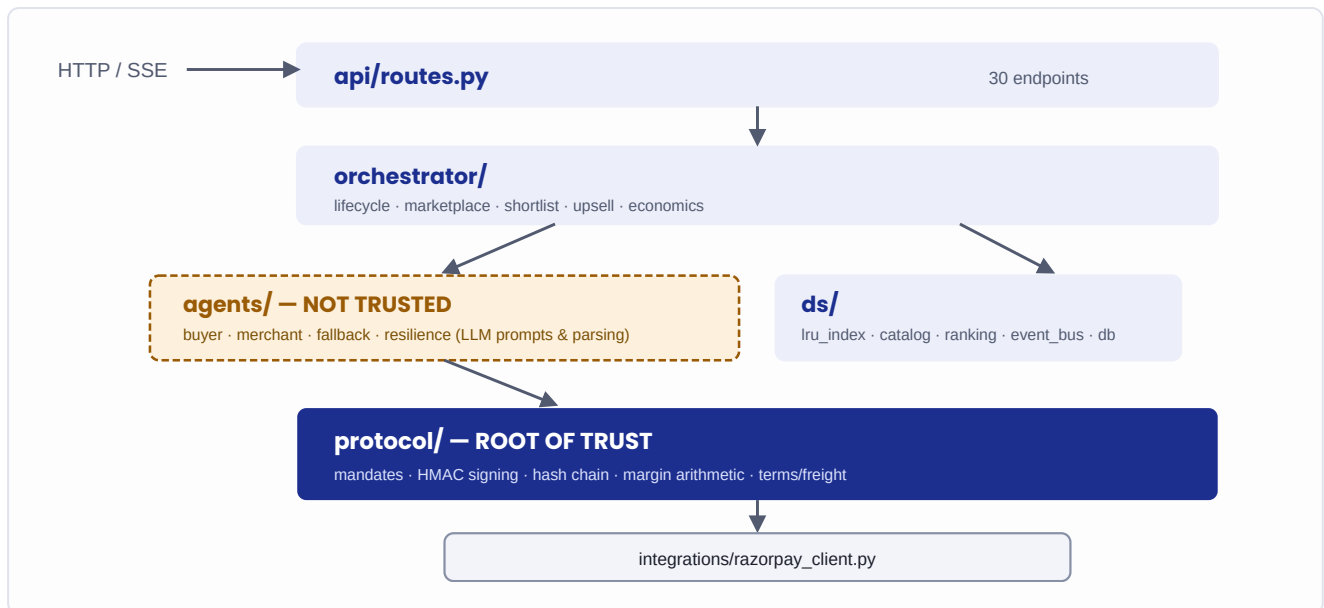


Fig. 1 — Layer stack and dependency direction. Solid blue = trusted; dashed amber = treated as adversarial. LLM output and the frontend are re-validated at every boundary they touch.

The LLM sits *inside* a box whose walls are all deterministic. It can choose **how** to negotiate; it cannot choose **whether** a bound applies. The same rule extends to the frontend — display only, with every constraint re-checked server-side.

03 The Protocol Layer — the Mandate Chain

Modelled on the AP2 / agent-payments idea of a chain of signed mandates, so each stage commits to the one before it.

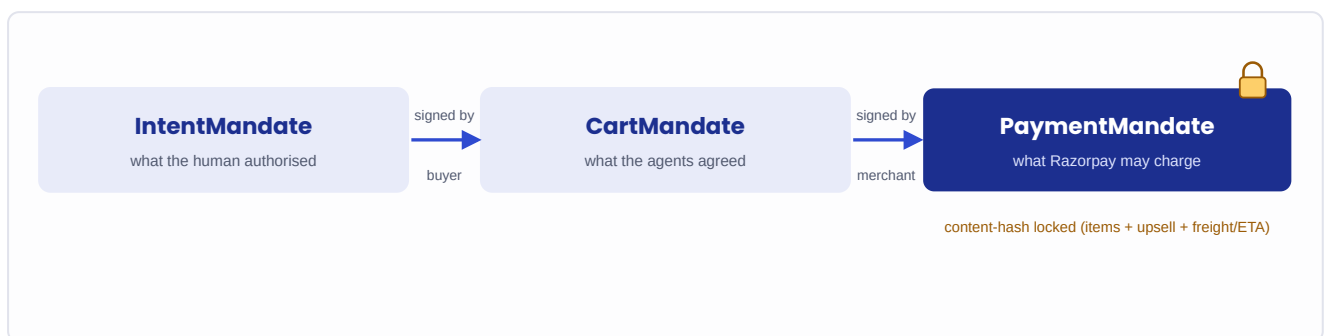


Fig. 2 — The signed mandate chain. Each stage's signature covers every bound the next stage must not exceed.

IntentMandate

Fields: goal, max_spend, qty_min/qty_max, ship_within_days, requested_lines[] (the named basket), preferred_payment_terms, and three priority weights. The signature covers every bound the agent must not exceed — budget, quantity band, delivery deadline, credit ceiling, **and the basket itself**. Signing only price and quantity would leave terms and goods unprotected: an approved mandate could then be settled on different delivery terms, credit, or an entirely different set of goods than the human saw.

CartMandate

Line items, an optional upsell line, lead time, payment terms, margin and reasoning — plus five derived totals (line total, goods subtotal, shipping cost, total price, net realisable total).

Bug found & fixed — these totals were declared as plain `@property`, which is invisible to `model_dump()`. Every persisted cart silently omitted its own totals; callers had to re-derive them, and the leaderboard was reading zero. Switched to `@computed_field`.

PaymentMandate & the hash lock

Before payment, the agreed cart is content-hashed: `hash_cart(items) = sha256(canonical_json({"items": [...]}))`. The payload includes every line item, the upsell line, and a synthetic line carrying freight and ETA — locking only line items would let a settled order be re-shipped on a slower, cheaper service without the lock noticing.

Two ordering bugs found & fixed — the cart was being signed *before* the upsell was attached (so the signature covered a total that no longer matched), and `total_price` wasn't being serialised at all. `POST /sessions/{id}/tamper` now mutates a locked cart and re-verifies, proving rejection — it returns `expected_hash`, `tampered_hash`, `rejected: true`.

Signing & the audit ledger

HMAC-SHA256 over canonical JSON. Per-agent secrets are *derived*, not hardcoded — `secret_for(agent_ref) = HMAC(MASTER_SECRET, agent_ref, sha256)` — so any newly registered business gets a valid, distinct signing identity with no dict to edit.

The ledger is append-only JSONL, one file per session, hash-chained: each entry's hash covers its own payload plus its predecessor's, so mutating or deleting any past entry breaks every hash after it. It stays JSONL rather than a database on purpose — a hash chain is the shape of an append-only file; SQLite would add a dependency and no extra integrity. Because the marketplace negotiates with several vendors in parallel threads, all appending to the same session ledger, read-last-hash-then-append is guarded by a lock — and critically, callers reuse **one AuditLedger instance per session across threads**, since a second instance would read a stale tail and corrupt the chain regardless of the lock. A typical session writes 50–75 entries; the README documents a one-line independent verifier that recomputes every hash from genesis without trusting the server's own flag.

04 The Commercial Model

This is the part most demos skip, and it's where the interesting bugs lived.

One definition of margin

There were previously two, silently compared against each other: pricing clamped to markup-on-cost (`cost × (1 + floor/100)`), while reporting measured margin-on-revenue (`(revenue - cost) / revenue`). A 12% floor clamped a ₹100-cost item to ₹112 — which reports as $12/112 = 10.71\%$, under its own floor, for every item, on every round. The clamp was correct and the measurement was correct; comparing them wasn't.

Impact — 14 spurious floor violations per session, only 1 of 3 vendors ever converging. After the fix: **0 violations, 3 of 3 converging**.

`margin_floor_pct` is named *margin*, so margin-on-revenue wins: `margin = (price - cost)/price` $\Rightarrow$ `price = cost/(1 - margin)`. Enforced by convention: never inline a margin formula — every caller goes through `min_sellable_price()` / `margin_pct()` from one shared module, so the two definitions cannot drift apart again.

Payment terms, lead time & freight — three priced levers

A rupee collected in 45 days isn't a rupee, and a floor checked against sticker price wouldn't notice a seller quietly conceding margin through terms instead. So **the floor is enforced on net realisable revenue** — what the seller actually banks, after financing cost — not sticker price.

Constant	Value	Rationale
<code>WORKING_CAPITAL_APR</code>	18.0%	Indian SMB working-capital lines run in the high teens
<code>MSMED_MAX_CREDIT_DAYS</code>	45	Legal ceiling (MSMED Act, s.43B(h)) — not a tuning knob
<code>ADVANCE_DISCOUNT_PCT</code>	2.0%	The classic "2/10" cash discount
<code>STANDARD_LEAD_DAYS</code>	7	Baseline delivery window
<code>BASE_FREIGHT_PER_UNIT</code>	₹11.0 (min ₹240)	ETA multipliers 1.00×–2.60×; long-window discount floors at 0.85×

Bug found & fixed — the lead-time discount was being applied *to the floor itself*, pushing the minimum below the floor — 16 violations and zero offers on any long-credit request.

Freight is priced as its own line and excluded from the margin the floor is enforced against — a merchant doesn't earn margin on someone else's diesel. Folding it into unit price (as an "expedite premium") was the earlier design, and no real quote works that way.

The upsell engine

Rule-based, not LLM — deliberately, so the decision is explainable and reproducible. It only offers an upsell when the base cart's margin already clears a 20-point headroom above the floor, the extra still fits inside the buyer's remaining budget, and the SKU isn't already in the cart.

05 Negotiation & the Multi-Vendor Marketplace

Up to six rounds per vendor; all vendors negotiate **concurrently** against one shared ledger. Three constraints are checked in code before the model is consulted at all:

Constraint	Failure mode prevented
<code>total_price > max_spend</code>	Overspend
<code>lead_time_days > ship_within_days</code>	A cart arriving after it's useful is a different deal, not a cheaper one
<code>basket_shortfall</code> non-empty	A vendor substituting whatever it's cheapest at for what was actually asked for

`basket_shortfall` matches on item name, case-folded, never SKU — each vendor sells the same goods under its own code. The upsell line is deliberately excluded from satisfying the basket: an extra the merchant attached can't count toward what the buyer asked for. Merchant proposals use JSON-schema structured output with the SKU list injected as an enum, so the model cannot name a product the vendor doesn't stock.

The shortlist gate — bounding spend, not just time

Negotiation costs ~9 model calls per vendor, so broadcasting to every merchant makes one purchase $O(\text{vendors})$ — at 100k merchants, ~900k model calls for a single restock. That's a **spend problem before it's a latency problem**, so the gate runs before the fan-out, keyed on a provable lower bound: the cheapest unit price a vendor could quote without breaching its own margin floor. No amount of negotiation can move it.

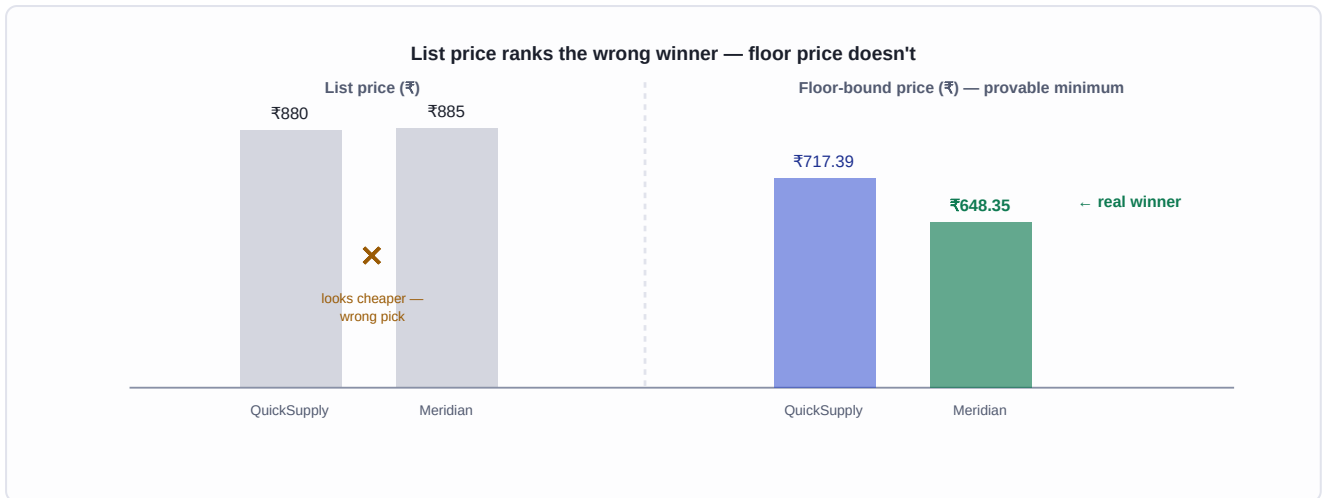


Fig. 3 — Real inversion from the seeded registry (Wireless Mouse). Ranking on sticker price picks QuickSupply; ranking on the provable floor bound correctly picks Meridian Supply.

Shortlist depth defaults to `SHORTLIST_K = 3`, making per-purchase spend **$O(1)$ in marketplace size**; the console reports what it saved (e.g. "Shortlisted 3 of 7 · saved ~48 model calls · ₹0.86"). A purely greedy shortlist means the same vendors win every auction — for a platform meant to grow merchant revenue, that's an anti-feature. An exploration slot for under-served vendors is implemented and unit-tested but defaults to 0: a deliberate, reversible one-constant choice that keeps today's shortlist exactly "the k cheapest achievable."

Scoring & human override

A multi-attribute score is weighted by the buyer's own stated priorities — price, speed, and terms.

Bug found & fixed — convergence speed was originally weighted as half of "speed," letting the most expensive vendor win purely for caving fastest. How quickly a vendor caved is a property of the negotiation, not of the deal — demoted to a 3% tiebreaker.

An offer below half the median of converged offers is flagged and its score discounted, so a gamed near-zero price can't win on raw cheapness. The scorer produces a **recommendation, not a verdict** — every converged cart stays selectable, and a human override is written to the ledger alongside the recommendation it replaced.

06 The Two Gates

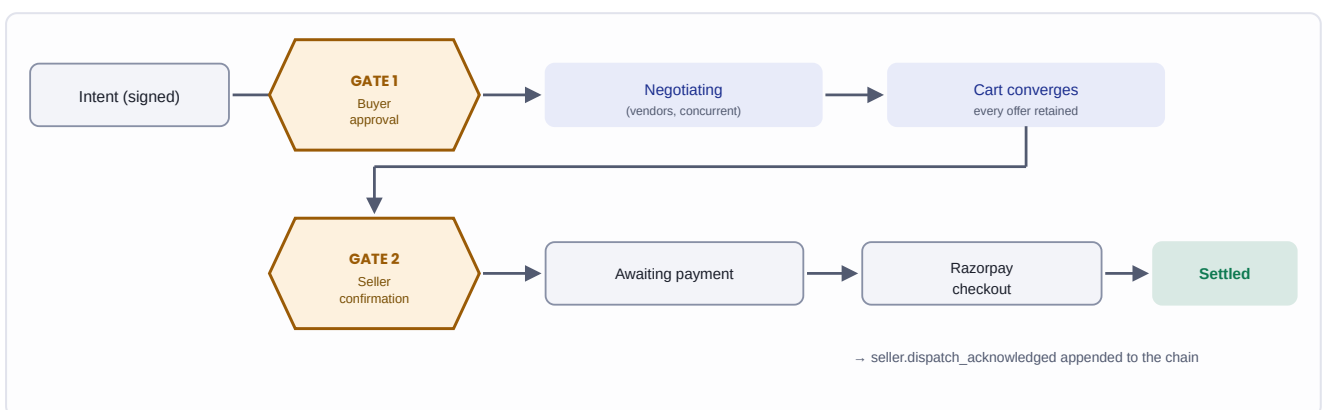


Fig. 4 — The lifecycle's two blocking gates. Nothing reaches an LLM, a vendor, or Razorpay before Gate 1; nothing is charged before Gate 2.

Gate 1 — buyer approval. Minting and signing an intent halts everything; nothing reaches an LLM, a vendor, or Razorpay until a human approval is recorded against the intent hash. There is no code path from an unapproved intent to a payment.

Gate 2 — seller confirmation, unconditional. Every converged cart parks for the merchant to accept. This was originally conditional on the margin being near the floor — optimising the wrong risk. The agent negotiated against a *catalog*, and a catalog is a claim about stock, not a fact; real merchants oversell and under-update. The seller can also decline — a gate that can only say yes isn't a gate — and every converged cart stays settleable, so the buyer takes the runner-up without a fresh negotiation.

Restart-durability bug found & fixed — confirmation originally read only an in-memory dict, so every deal parked before a process restart was permanently unconfirmable — **274 sessions were stuck**. Rebuilding the CartMandate from the persisted session reproduces the identical hash and signature, since both are functions of field values, not object identity.

07 Resilience

Retries distinguish transient failures (dropped socket, 429, 5xx — worth retrying) from terminal ones (malformed request, auth failure — propagate immediately). Backoff is bounded and **fully jittered** on purpose: all vendor threads fail simultaneously on a network blip, and without jitter they'd retry in lockstep and hammer the same recovering endpoint.

When retries are exhausted, a rule-based fallback builds a valid cart with **no model at all** — filling the quantity band from the vendor's catalog, pricing at a round-appropriate margin, walking price toward the floor if over budget, and trimming quantity only as a last resort, never below what the buyer explicitly named and never below the floor. Every degraded cart is flagged explicitly — on the cart, in the UI, and in the audit trail — so "not degraded" is a positive assertion, not an absence.

A chaos switch ([POST /chaos/model-outage](#)) makes the model genuinely unreachable rather than jumping straight to the fallback: it raises the same connection error the SDK would raise, so failure classification, rule-based quoting and degraded labelling all execute exactly as in a real outage. It self-expires after 300s.

Verified with the model fully dead — all 3 shortlisted vendors degraded at round 1, a winner was still selected, a real Razorpay order was created, margin landed at **14.99%** against the winning vendor's **9%** floor, and the chain stayed valid.

Failure	Response	Resulting state
OpenAI unreachable	Rule-based quote after 3 tries	<code>degraded: true</code> , order still completes
Razorpay order/capture fails	Mandate held intact, bounded 30s	<code>payment_provider_error</code>
Bad checkout signature	HMAC verify, refuse capture	<code>payment_signature_invalid</code>
Tampered cart	Hash mismatch, refuse settlement	<code>rejected_hash_mismatch</code>
Budget / basket infeasible	Rejected before any model call, with arithmetic	400 + explanation
Process restart mid-deal	Rehydrate cart from persisted session	Confirmable

08 Data Structures, Scale & Security

Structure	Chosen for	Complexity
SessionStore	LRU (512 hot) + inverted indices over a disk-backed corpus	$O(1)$ get/put; $O(k)$ by-buyer/by-seller
CatalogIndex	Trie + inverted index — catalog agent-readable at scale	$O(\text{len}(\text{prefix}) + \text{matches})$ prefix; $O(\text{rarest posting list})$ term search
top_k_offers	One-shot top-k over a batch	$O(n \log k)$ bounded heap
Leaderboard	Incrementally maintained ranking (no decrease-key on a heap)	$O(\log n)$ write via bisect.insort ; $O(1)$ read
EventBus	Ring buffer + per-subscriber queues so SSE isn't a black box for 10–20s	Bounded memory; publish never blocks

Sorting is microseconds at any scale tested — the wall is always the **model calls**, which is why the shortlist gate, not the heap, is the line that matters. It's framed as a spend bound, not a speed optimisation.

Scale (N merchants)	Bottleneck	Mitigation
$\sim 10^3$	In-memory registry; shortlist scans all vendors for bounds	Move registry to SQLite; pre-compute bounds into the catalog index
$\sim 10^4$ – 10^5	Single-process assumption; in-memory pending/converged caches; SSE fan-out	Externalise to Redis/DB; Redis pub/sub or NATS for the bus
$\sim 10^6$	SQLite write concurrency; single-node ledger	Postgres; shard ledgers by session; cold audit to object storage

Security posture

Implemented: HMAC-signed mandates with derived per-agent keys; hash-chained tamper-evident ledger; content-hash lock over items and freight/ETA; Razorpay checkout signature verification; structured output with SKU enums so the model can't invent products; all bounds re-validated server-side; cost data never exposed to agents.

Explicitly not production-grade, labelled as such in code: a single master signing secret rather than per-agent keys issued at registration; no authentication, authorisation, or rate limiting on any endpoint; a deliberately credential-free merchant identity switch for demonstration; simulated route attribution.

09 Verification & Results

- **Chain verification** — every hash recomputed from genesis independently, not trusting the server's own flag. Valid across every session tested.
- **Floor invariant** — margin-floor violations at 0 across sessions after the margin fix (was 14/session).
- **Tamper** — mutation of a locked cart produces a different hash and is rejected.
- **Degraded end-to-end** — model fully dead; order still completes on a real Razorpay order; floor still enforced; every cart flagged.
- **Restart durability** — a previously stuck parked deal confirmed after a restart, producing a real order.
- **Full payment lifecycle** — a real Razorpay test-mode payment captured ([pay_TYGuNpzPXHh6Eu](#) , ₹23,672.75), dispatch acknowledged by the seller, chain still valid.

- **Full UI pass** — vendor creation, multi-item negotiation, human override, merchant acceptance, tamper, degraded run, across all five routes: zero console errors, zero backend tracebacks, zero 5xx.
- **Clean-clone install** — verified twice: with credentials, and with **none at all** (intent minted, gate held, rule-based path, 45-entry chain valid).

10 Design Judgment & Honest Gaps

Decision	Reasoning
SQLite, not a hosted DB	The submission is a repo link; a clean clone must run with no external service or credentials
Audit ledger stays JSONL	A hash chain <i>is</i> an append-only file; a DB adds a dependency and no extra integrity
Chaos switch breaks the upstream	Short-circuiting straight to the fallback would demonstrate nothing
Seller gate unconditional	Catalogs are claims about stock, not facts
Floor on net realisable revenue	Otherwise payment terms become a hidden margin leak
No public deployment	A 40–70s SSE stream needs a persistent container; mitigated by a fully credential-free clean-clone path

Honest gaps

Stated plainly, because a reviewer will find them anyway:

1. Revenue figures are measured against seeded vendors and the author's own test-mode runs, not live merchant traffic — never presented as forecasts.
2. No public deployment; mitigated rather than solved — a clean clone runs the whole system with zero credentials, every cart labelled degraded, and adding keys shows it live.
3. Two of three in-memory caches remain restart-fragile — only pending-confirmation rehydration is fixed so far.
4. Economics counters reset on restart — sessions persist via SQLite, token counters don't.
5. A clean clone starts at zero sessions, so live counters read empty until the first negotiation runs.

11 Mapping to the Brief

"Every money action explainable, bounded and gated. Show the audit trail and one failure handled gracefully."

Requirement	Where
Transactable by an AI buyer end to end	Intent → shortlist → negotiate → seller accept → Razorpay order → payment → dispatch
Agent-readable catalog (<i>named direction</i>)	Agent card, trie + inverted index, cross-vendor search
Upsell & cross-sell agent (<i>named direction</i>)	Rule-based upsell engine
Gated	Two blocking human gates; the seller can accept or decline
Bounded	Margin floor, basket, budget, MSMED cap — all arithmetic
Explainable	Hash-chained ledger, independently verifiable
Audit trail shown	/audit , live trail, README verifier
One failure handled gracefully	Chaos switch → real outage → real fallback → floor holds

Mandate — Technical Breakdown · Dev Chalana · AI Growth & Agentic Commerce